

1 Einleitung

1.1 Ziele

Das Ziel dieser Diplomarbeit ist es, ein wartbares, erweiterbares und einfach aufsetzbares Zeitmesssystem für die *Zero Emission Challenge* zu entwickeln, welches den gesamten Ablauf von der Zeiterfassung über die Punkteausswertung bis zur Ergebnisanzeige umfasst. Das System soll zusätzlich auch die Verwaltung von Teams und Benutzern ermöglichen und eine rollenbasierte Zugriffskontrolle implementieren.

Das entwickelte System soll:

- Renndaten aller Bewerbe zuverlässig erfassen, speichern und auswerten
- Versuchsdaten per Application Programming Interface (API) in das System speichern zu können
- Eine webbasierte Plattform zur Verwaltung von Teams, Benutzern und Ergebnissen bereitstellen
- Sicherheit durch rollenbasierte Zugriffskontrolle und eine zentrale Authentifizierung über Keycloak gewährleisten
- Mittels Docker einfach und plattformunabhängig betreibbar sein
- Softwarequalität durch automatisierte Tests wie Pytest und Playwright sicherstellen

1.2 Individuelle Aufgabenstellung

Um diesen Zielen gerecht zu werden, wurden folgende Aufgaben definiert, die im Rahmen dieser Diplomarbeit umgesetzt werden sollen:

- Entwicklung eines in Microservices aufgeteilten RESTful-API-Backends zur Zeiterfassung, Punkteausswertung sowie Team- und Benutzerverwaltung
- Entwicklung einer webbasierten Anwendung mit unterschiedlichen Funktionen für die Benutzergruppen Administrator, Teamleiter und Zuschauer
- Implementierung eines dedizierten Authentifizierungsdienstes
- Rollenbasierte Zugriffskontrolle auf die Website sowie die API-Endpunkte
- Modellierung und Anbindung einer relationalen PostgreSQL-Datenbank zur Datenspeicherung
- Containerisierung aller Systemkomponenten mittels Docker
- Implementierung automatisierter Testfälle mit Pytest und Playwright

2 Theoretische Grundlagen

2.1 Containerisierung

Containerisierung ist eine Methode, bei der Anwendungen zusammen mit ihren Abhängigkeiten und Konfigurationsdateien in isolierten Umgebungen, sogenannten Containern, ausgeführt werden. Traditionell muss man, um eine Anwendung zu installieren, die zum jeweiligen Betriebssystem kompatible Version herunterladen. Mit Containerisierung ist es jedoch möglich, das gesamte Softwarepaket in einem Container auf verschiedenen Betriebssystemen auszuführen[1]

2.1.1 Docker

Docker ist eine weitverbreitete, offene Plattform zur Erstellung und Verwaltung von Containern. Docker ermöglicht es, Anwendungen von der Infrastruktur zu trennen, indem es Container bereitstellt, die in jeder Umgebung konsistent laufen.[2]

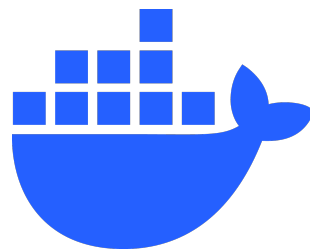


Bild 1: Docker Logo

Quelle: <https://www.docker.com/company/newsroom/media-resources/>

2.1.2 Architektur von Docker

Docker verwendet eine Client-Server-Architektur. Der *Docker-Client* kommuniziert dabei mit dem *Docker-Daemon* (`dockerd`), welcher die eigentliche Arbeit des Erstellens, Ausführens und Verteilens von Containern übernimmt. Die Kommunikation zwischen Client und Daemon erfolgt über eine REST-API, wahlweise über Unix-Sockets oder eine Netzwerkschnittstelle.[2]

2.1.3 Funktionsweise

Alles beginnt mit einem Dockerfile. Docker erstellt Images basierend auf den Anweisungen in einem Dockerfile. Ein Dockerfile ist eine Textdatei, die die Anweisungen enthält, wie der Quellcode gebaut wird, welche Pakete installiert werden müssen und wie die Anwendung gestartet wird. Docker-Images bestehen aus mehreren Schichten (Layers). Jede Anweisung im Dockerfile erzeugt eine neue Schicht, die auf den vorherigen aufbaut.[3]

Ein Container ist eine laufende Instanz eines Docker-Images. Er enthält alles, was die Anwendung benötigt, um zu laufen, einschließlich des Programmcodes. Docker-Container sind leichtgewichtig, da sie den Betriebssystemkernel des Host-Systems gemeinsam nutzen, anstatt wie virtuelle Maschinen ein ganzes Betriebssystem zu benötigen.[2]

2.1.4 Docker Compose

Docker Compose ist ein Werkzeug zur Definition und Verwaltung von Multi-Container-Applikationen. In einer einzigen YAML-Datei `compose.yaml`, werden alle Komponenten einer Applikation beschrieben.[4] In dieser Diplomarbeit (DA) wird Docker Compose verwendet, um Backend-Services, Datenbanken, API-Gateway sowie den Authentifizierungsdienst mit einem einzigen Befehl starten zu können.

2.1.5 Vorteile der Containerisierung

Der Hauptvorteil der Containerisierung liegt in der Eliminierung des „*works on my machine*“-Problems. Ein Image, das einmal erstellt wurde, läuft auf jedem System identisch, unabhängig von der Infrastruktur, egal ob es sich um einen Raspberry Pi oder einen Server handelt. Da jeder Container komplett isoliert ist, eignet sich die Containerisierung auch sehr gut für die Microservice-Architektur, da jeder Service in einem eigenen Container laufen kann, ohne dass es zu Konflikten mit Abhängigkeiten anderer Services kommt. Das erleichtert sowohl die Entwicklung als auch Deployment und Betrieb, da das System immer identisch läuft. Weiters ermöglichen die Container eine leichte horizontale Skalierbarkeit. Mittels Orchestrierungstools wie Kubernetes können Container automatisch skaliert und verwaltet werden, um den Anforderungen der Anwendung gerecht zu werden.

2.2 Microservices

Die Microservice-Architektur ist ein Ansatz zur Softwareentwicklung, bei dem eine Applikation als Sammlung unabhängiger Komponenten aufgebaut wird, wobei jede als eigenständiger Service läuft. Diese Services kommunizieren über definierte Schnittstellen mittels APIs. Jeder Service ist auf eine spezifische Aufgabe ausgerichtet und kann unabhängig von den anderen aktualisiert, deployt und skaliert werden.[5]

2.2.1 Vorteile von Microservices

- **Flexible Skalierung:** Jeder Service kann unabhängig von den anderen skaliert werden, um der Nachfrage gerecht zu werden, ohne das gesamte System zu skalieren.
- **Technologische Freiheit:** Teams können für jeden Service die passendste Technologie wählen, unabhängig von den Technologie-Stacks anderer Services. Zum Beispiel könnte ein neuer Service für diese DA in Rust implementiert werden, während sich die anderen Services nicht ändern müssen.
- **Einfaches Deployment:** Änderungen an einem Service können unabhängig deployt werden, ohne das gesamte System neu zu deployen. Das ermöglicht schnellere Iterationen und häufigere Updates.
- **Resilienz:** Falls ein einzelner Service ausfällt, kann die Anwendung weiterhin funktionieren, wenn auch mit eingeschränkter Funktionalität.[5]

2.2.2 Nutzung von Microservices in dieser DA

In dieser DA wurde der Backend-API-Server als Microservice-Architektur umgesetzt. Er ist in mehrere unabhängige Services unterteilt, die jeweils über eine eigene Datenbankinstanz verfügen und über REST-APIs miteinander kommunizieren. Alle Anfragen der Webanwendung werden über ein zentrales Nginx-API-Gateway geleitet, welches die Anfragen an den jeweils zuständigen Service weiterleitet.

3 Technologieauswahl

3.1 GitHub

Als Versionskontrollplattform wurde GitHub gewählt, welches auf dem verteilten Versionskontrollsystem Git basiert und kollaborative Softwareentwicklung durch Branching, Pull Requests und Code-Reviews ermöglicht.[6] Im Vergleich zu Alternativen wie GitLab oder Bitbucket bietet GitHub die größte Entwickler-Community und eine nahtlose Integration mit zahlreichen Entwicklungstools.

3.2 Nginx

Als API-Gateway wurde Nginx gewählt, welches eingehende HTTP-Anfragen entgegennimmt, die JSON Web Token (JWT)-Authentifizierung über das `auth_request`-Modul übernimmt und Anfragen an den jeweils zuständigen Microservice weiterleitet.[7] Im Vergleich zu Alternativen wie Traefik oder HAProxy bietet Nginx eine besonders hohe Stabilität sowie eine flexible Konfigurierbarkeit. Gegenüber einem vollwertigen API-Gateway wie Kong bringt Nginx deutlich weniger Overhead mit und ist damit besser für kleine Microservice-Deployments geeignet. In der DA übernimmt Nginx die zentrale Rolle als einziger Einstiegspunkt für alle Anfragen der Webanwendung und stellt sicher, dass kein Service ohne gültige Authentifizierung erreichbar ist.

3.3 Docker

Zur Containerisierung wurde Docker gewählt, welches, wie in Kapitel 2.1 detailliert beschrieben, die plattformunabhängige Bereitstellung aller Systemkomponenten ermöglicht. [2] Im Vergleich zu alternativen Container-Runtimes wie Podman ist Docker durch seine weite Verbreitung und die umfangreiche Dokumentation besonders gut geeignet. Die Containerisierung ermöglicht das Hosting aller Services auf unterschiedlichen Plattformen, von einem Raspberry Pi bis hin zu einem Server, ohne dass Änderungen am Code notwendig sind.

3.4 PostgreSQL

Für die Datenhaltung wurde PostgreSQL als relationale Datenbank gewählt, welche ACID-Konformität, komplexe Abfragen und strukturierte Datenbeziehungen unterstützt.[8] Während NoSQL-Datenbanken wie MongoDB für dokumentenbasierte oder hochskalierbare Anwendungen Vorteile bieten, fällt die Entscheidung aufgrund der strukturierten Beziehungen zwischen den Daten im vorliegenden Fall auf eine relationale Datenbank. Gegenüber MySQL bietet PostgreSQL eine strengere Standardkonformität sowie bessere Unterstützung für komplexe Datentypen und Constraints, was die Datenintegrität über alle Microservices hinweg sicherstellt.

3.5 Keycloak

Für die Authentifizierung und Benutzerverwaltung wurde Keycloak als dedizierter Identity Provider gewählt, welcher OAuth2- und OpenID-Connect-basierte Authentifizierung sowie rollenbasierte Zugriffskontrolle (RBAC) out-of-the-box bereitstellt.[9] Im Vergleich zur Eigenimplementierung reduziert Keycloak den Entwicklungsaufwand erheblich und bietet gleichzeitig ein höheres Sicherheitsniveau durch standardkonforme Protokolle. Gegenüber einfacheren Lösungen wie Auth0 bietet Keycloak den Vorteil, dass es vollständig mit Docker selbst gehostet werden kann. In der DA übernimmt Keycloak die Ausstellung und Validierung von JWTs.

3.6 SQLAlchemy

Als Object-Relational-Mapper (ORM) wurde SQLAlchemy gewählt, welches den Zugriff auf die PostgreSQL-Datenbank über Python-Objekte abstrahiert und direkte Structured Query Language (SQL)-Abfragen überflüssig macht.[10] Im Vergleich zu reinen SQL-Ansätzen reduziert SQLAlchemy Boilerplate-Code erheblich, verbessert die Lesbarkeit und ermöglicht durch das deklarative Modellsystem eine klare Abbildung der Datenbankstruktur auf Python-Klassen.

3.7 FastAPI

Als Web-Framework wurde FastAPI gewählt. FastAPI ist ein modernes Python-Web-Framework, welches automatisch API-Dokumentation generiert.[11] Im Vergleich zu anderen Python-API-Frameworks wie Flask bietet FastAPI native Type-Hints-Unterstützung und automatische Validierung von Request- und Response-Daten durch Pydantic.[12] Verglichen mit Django bringt FastAPI deutlich weniger Overhead mit und ist damit besser für kleine Micro-services geeignet.[12] In der DA bildet FastAPI die Grundlage jedes einzelnen Backend-Services und stellt die jeweiligen REST-Endpunkte zur Verfügung, über die die Webanwendung und die Services untereinander kommunizieren.

3.8 Next.js

Als Frontend-Framework wurde Next.js gewählt, welches auf React basiert und sowohl serverseitiges Rendering (SSR) als auch statische Seitengenerierung (SSG) unterstützt.[13] Im Vergleich zu anderen Frameworks spricht für Next.js die bessere Ladezeit sowie die vereinfachte Projektstruktur durch das dateibasierte Routing-System. In der DA wird Next.js für die Webanwendung eingesetzt, über die die Nutzer auf das System zugreifen.

3.9 shadcn/ui

Als UI-Bibliothek wurde shadcn/ui gewählt, welches auf Radix UI und Tailwind CSS basiert und vorgefertigte, vollständig anpassbare Komponenten bereitstellt.[14] Gegenüber einer Selbstimplementierung reduziert shadcn/ui den Programmieraufwand für häufig benötigte UI-Elemente wie Formulare, Dialoge und Tabellen erheblich, ohne dabei die Flexibilität beim Styling einzuschränken.

4 Systemarchitektur

4.1 Überblick

Das Zero Emission Challenge (ZEC)-Timingsystem besteht aus zwei zentralen Teilen: einer webbasierten Anwendung, welche zur Anzeige der Renndaten und zur Verwaltung der Teams und Fahrer dient, sowie einem Backend-Server, welcher als Microservice-Architektur umgesetzt ist. Der Server übernimmt die gesamte Businesslogik, die Authentifizierung, die Datenverwaltung und die Kommunikation mit der Webanwendung und anderen externen Systemen. Abbildung 2 zeigt einen Überblick über alle Komponenten des Servers und deren Zusammenspiel intern sowie nach außen.

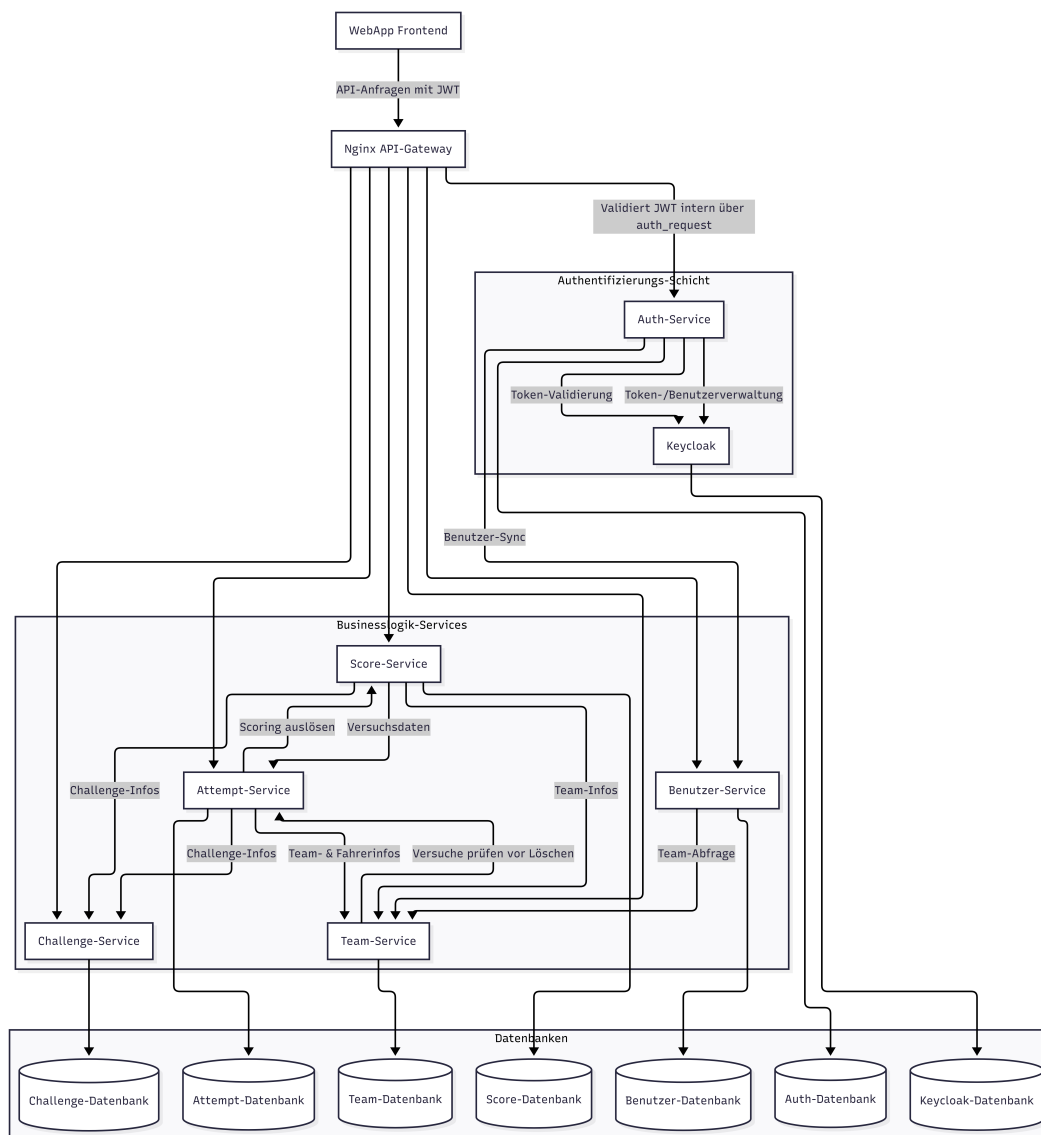


Bild 2: Systemarchitektur

4.2 Server

Die Microservices kommunizieren untereinander über API, wenn sie Daten von anderen Services benötigen. Braucht z.B. der Attempt Service Informationen über ein Team, fragt er diese beim Team Service ab. Jeder Service verwaltet seine Daten in seiner eigenen PostgreSQL-Datenbank. Da jeder Service eine eigene Datenbank besitzt, existieren die im Gesamtschema 3 dargestellten Beziehungen zwischen den Tabellen nicht als echte Fremdschlüssel, sondern werden in den Services sichergestellt.

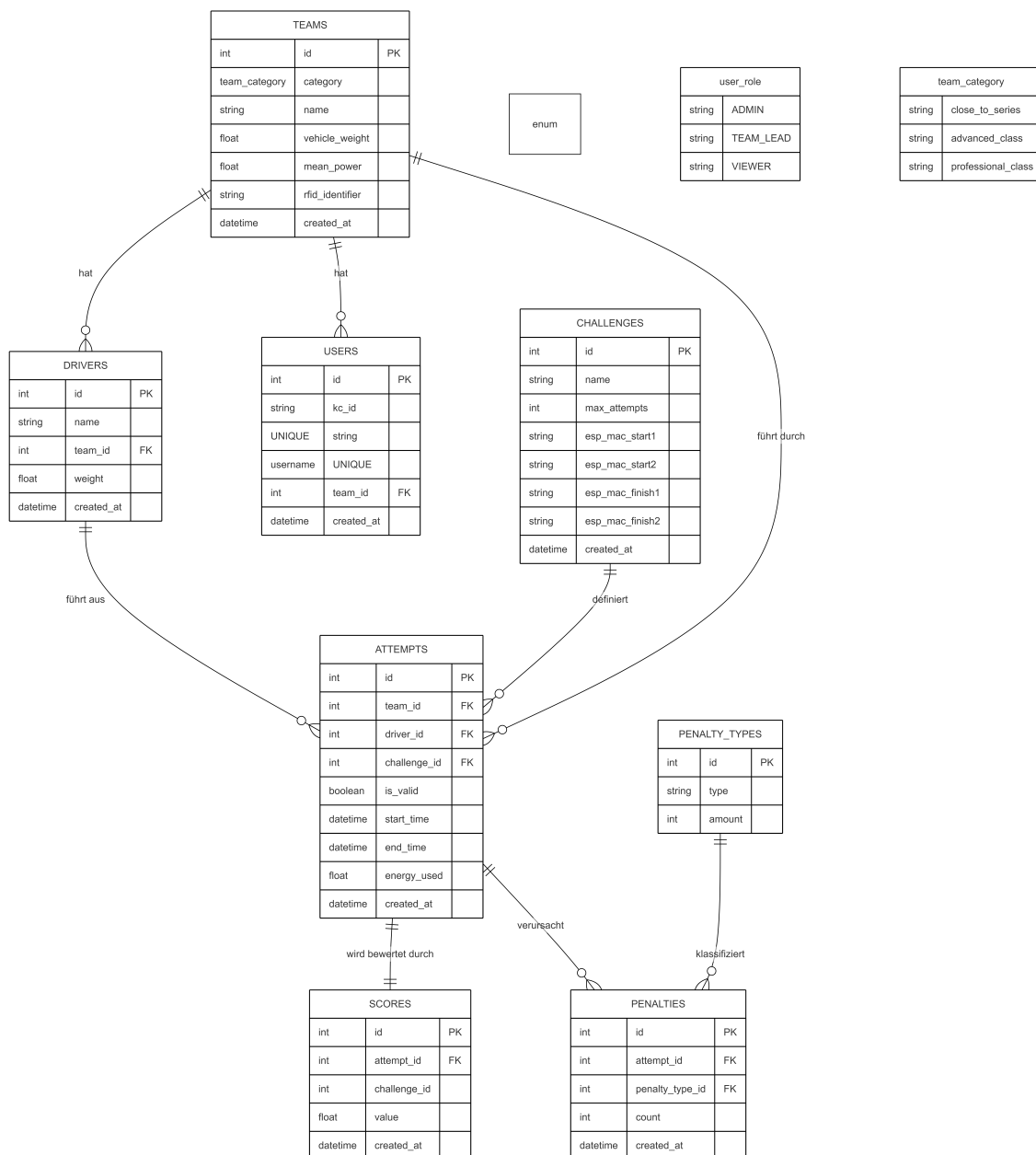


Bild 3: Datenbankschema des ZEC-Timing-Systems

4.2.1 Nginx API-Gateway

Das API-Gateway bildet den einzigen Zugang zum System und ist für das Routing, die Authentifizierung sowie die CORS-Konfiguration aller Anfragen zuständig.

Routing

Das Routing erfolgt pfadbasiert; jeder Pfad wird einem bestimmten Service zugeordnet:

- `/users` → User Service
- `/teams, /drivers` → Team Service
- `/challenges` → Challenge Service
- `/attempts` → Attempt Service
- `/scores, /leaderboard, /penalties` → Score Service
- `/login, /refresh` → Auth Service

Authentifizierung

Bei jeder weitergeleiteten Anfrage wird ein Subrequest an den Auth Service gesendet, welcher den angehängten JWT validiert und die Rolle des Users prüft. Schlägt die Validierung fehl, wird die Anfrage abgewiesen, ohne die anderen Services zu treffen. Anfragen mit einem gültigen API-Key bilden eine Ausnahme, sie sind für die Kommunikation mit anderen Applikationen im System vorgesehen.

Header-Weitergabe

Nach erfolgreicher Authentifizierung fügt Nginx die Userdaten als HTTP-Header der weitergeleiteten Anfrage hinzu. Services können so auf die Identität des Nutzers zugreifen, ohne Tokenvalidierung durchführen zu müssen.

CORS

Für alle Endpunkte werden die CORS-Header gesetzt, sodass die Webanwendung vom Browser aus auf die API zugreifen kann. Preflight-Anfragen werden direkt von Nginx beantwortet, ohne dass ein Backend-Service involviert wird.

4.2.2 Auth Service

Der Auth Service übernimmt jegliche Authentifizierung im System und ist eng mit Keycloak als Identity Provider verbunden.

Interne Verifikation

Der Auth Service stellt drei interne Endpunkte bereit, welche ausschließlich vom API-Gateway aufgerufen werden. Es gibt pro Nutzerrolle (Administrator, Teamleiter, Viewer) einen Endpunkt, welcher überprüft, ob der Nutzer die entsprechende Rolle besitzt.

Login & Token-Refresh

Öffentlich stellt der Auth Service zwei Endpunkte bereit: einen Login-Endpunkt und einen Refresh-Endpunkt. Diese dienen der Authentifizierung sowie der Erneuerung eines abgelaufenen Tokens.

4.2.3 User Service

Der User Service verwaltet die Nutzerkonten außerhalb von Keycloak und agiert als Bindeglied zwischen Keycloak und der restlichen Anwendung. Er speichert nur systeminterne Daten ; Passwörter und Rollen liegen jedoch in Keycloak.

Datenbankschema

Spalte	Typ	Beschreibung
id	Integer (PK)	Interne ID des Benutzers
kc_id	String (UNIQUE)	Keycloak-ID des Benutzers
username	String (UNIQUE)	Benutzername
team_id	Integer	Zugehöriges Team (optional)
created_at	DateTime	Erstellungszeitpunkt

Tabelle 1: Datenbankschema: User Service

Die `kc_id` stellt die Verbindung zwischen dem lokalen Datenbankdatensatz und dem entsprechenden Keycloak-Benutzerkonto her. Wie in 4.2 beschrieben, ist `team_id` kein echter Fremdschlüssel, sondern wird in der Businesslogik des User Service sichergestellt.

Funktionalität

Der User Service ermöglicht das Erstellen, Aktualisieren und Löschen von Benutzern, wobei Änderungen stets sowohl in Keycloak als auch in der lokalen Datenbank durchgeführt werden. Darüber hinaus können Benutzern Rollen in Keycloak zugewiesen oder entzogen werden. Für Operationen in Keycloak bezieht der User Service über den Auth Service einen kurzlebigen Admin-Token.

4.2.4 Team Service

Der Team Service verwaltet die teilnehmenden Teams sowie deren Fahrer und stellt diese Daten den anderen Services über seine REST-API zur Verfügung.

Datenbankschema

Spalte	Typ	Beschreibung
id	Integer (PK)	Interne ID des Teams
name	String	Name des Teams
category	Enum	Teamkategorie
vehicle_weight	Float	Fahrzeuggewicht in kg
mean_power	Float	Durchschnittliche Leistung in W
rfid_identifizier	String	RFID-Kennung des Fahrzeugs
created_at	DateTime	Erstellungszeitpunkt

Tabelle 2: Datenbankschema: Teams

Spalte	Typ	Beschreibung
id	Integer (PK)	Interne ID des Fahrers
name	String	Name des Fahrers
team_id	Integer	Zugehöriges Team
weight	Float	Gewicht des Fahrers in kg
created_at	DateTime	Erstellungszeitpunkt

Tabelle 3: Datenbankschema: Drivers

Funktionalität

Der Team Service bietet die Funktionalität zum Erstellen, Lesen, Aktualisieren und Löschen von Teams und Fahrern. Greift ein User mit der Rolle TEAM_LEAD auf die API zu, wird überprüft, ob er auf sein eigenes Team zugreift, andernfalls wird die Anfrage abgelehnt.

4.2.5 Challenge Service

Der Challenge Service verwaltet die einzelnen Bewerbe der ZEC und stellt deren Konfiguration den anderen Services zur Verfügung. Ein Bewerb definiert die maximale Anzahl an Versuchen sowie die MAC-Adressen der zugehörigen Lichtschranken.

Datenbankschema

Spalte	Typ	Beschreibung
id	Integer (PK)	Interne ID des Bewerbs
name	String	Name des Bewerbs
max_attempts	Integer	Maximale Anzahl an Versuchen
esp_mac_start1	String	MAC-Adresse Startlichtschranke 1
esp_mac_start2	String	MAC-Adresse Startlichtschranke 2
esp_mac_finish1	String	MAC-Adresse Ziellichtschranke 1
esp_mac_finish2	String	MAC-Adresse Ziellichtschranke 2
created_at	DateTime	Erstellungszeitpunkt

Tabelle 4: Datenbankschema: Challenge Service

Funktionalität

Der Challenge Service bietet nur Leseoperationen und Aktualisierungen an, da die Bewerbe einmalig per Datenbankskript angelegt und danach nur noch aktualisiert werden. GET-Anfragen sind öffentlich zugänglich, sodass die Ergebnisanzeige ohne Anmeldung auf Bewerbungsinformationen zugreifen kann. Schreibende Operationen sind ausschließlich Administratoren erlaubt.

4.2.6 Attempt Service

Der Attempt Service ist für die Erfassung und Verwaltung der Versuche zuständig. Er hat eine Vielzahl von Inter-Service-Abhängigkeiten, siehe 2, da viele Daten verifiziert werden müssen. Außerdem werden automatisch die Punkteberechnung sowie die Strafpunkte im Score Service ausgelöst.

Datenbankschema

Spalte	Typ	Beschreibung
id	Integer (PK)	Interne ID des Versuchs
team_id	Integer	Zugehöriges Team
driver_id	Integer	Zugehöriger Fahrer
challenge_id	Integer	Zugehöriger Bewerb
is_valid	Boolean	Gültigkeit des Versuchs
start_time	DateTime	Startzeitpunkt
end_time	DateTime	Endzeitpunkt
energy_used	Float	Verbrauchte Energie in Wh
created_at	DateTime	Erstellungszeitpunkt

Tabelle 5: Datenbankschema: Attempt Service

Funktionalität

Beim Erstellen eines Versuchs werden Team, Fahrer und Bewerb validiert sowie überprüft, ob das Team die maximale Versuchsanzahl bereits erreicht hat. Nach dem Speichern werden optional Strafpunkte sowie automatisch ein Score-Eintrag im Score Service angelegt. Wird ein Versuch nachträglich als ungültig markiert, wird der zugehörige Score automatisch entfernt. Beim Löschen eines Versuchs werden Score und Strafpunkte ebenfalls gelöscht.

Export

Über einen Export-Endpoint können alle Versuche eines Bewerbs als CSV- oder XLSX-Datei heruntergeladen werden. Die Daten werden dabei mit Team- und Fahrerinformationen verbunden und können optional nach Teamkategorie gefiltert werden.

4.2.7 Score Service

Der Score Service ist für die Berechnung und Verwaltung der Punktestände sowie die Bereitstellung der Ergebnislisten zuständig. Er verwaltet neben den Scores auch die Strafpunkte und implementiert die bewerbungsspezifische Punkteberechnungslogik.

Datenbankschema

Spalte	Typ	Beschreibung
id	Integer (PK)	Interne ID des Scores
attempt_id	Integer	Zugehöriger Versuch
challenge_id	Integer	Zugehöriger Bewerb
value	Float	Berechneter Punktwert
created_at	DateTime	Erstellungszeitpunkt

Tabelle 6: Datenbankschema: Scores

Spalte	Typ	Beschreibung
id	Integer (PK)	Interne ID der Strafe
attempt_id	Integer	Zugehöriger Versuch
penalty_type_id	Integer	Art der Strafe
count	Integer	Anzahl der Strafen
created_at	DateTime	Erstellungszeitpunkt

Tabelle 7: Datenbankschema: Penalties

Spalte	Typ	Beschreibung
id	Integer (PK)	Interne ID des Straftyps
type	String	Bezeichnung der Strafe
amount	Integer	Strafzeit in Sekunden

Tabelle 8: Datenbankschema: Penalty Types

Punkteberechnung

Die Punkteberechnung ist pro Bewerb unterschiedlich. Die Berechnungsformeln der einzelnen Bewerbe werden im Kapitel zur Score-Implementierung im Detail beschrieben.

Leaderboard

Pro Bewerb und Teamkategorie gibt es eine Ergebnisliste, in der jedes Team mit seinem besten Score angeführt wird. Diese Listen können als CSV- oder XLSX-Datei exportiert werden. GET-Anfragen auf die Leaderboard-Endpunkte sind öffentlich zugänglich, sodass die Ergebnisanzeige ohne Anmeldung auf die Daten zugreifen kann.

4.3 Web App

Die Webanwendung ist mit Next.js[13] umgesetzt und bildet die Benutzeroberfläche des ZEC-Timing-Systems. Sie kommuniziert über das Nginx API-Gateway mit dem Backend und passt ihre Anzeige dynamisch an die Rolle des angemeldeten Benutzers an.

Authentifizierung & Sitzungsverwaltung

Die Anmeldung erfolgt über den Login-Tab, auf dem Benutzername und Passwort eingegeben werden. Nach erfolgreicher Anmeldung werden Access- und Refresh-Token lokal gespeichert und bei jeder API-Anfrage mitgesendet. Läuft die Sitzung ab, wird der Benutzer automatisch zur Login-Seite weitergeleitet und über das Sitzungsende informiert.

Rollenbasierte Ansichten

Die Webanwendung unterscheidet drei Benutzerrollen mit unterschiedlichen Tabs:

- **Viewer:** Hat nur Zugriff auf die öffentliche Ergebnisliste (*Leaderboard*), die die aktuellen Punktestände aller Teams pro Bewerb und Teamkategorie anzeigt. Dieser Bereich ist auch ohne Anmeldung zugänglich.
- **Teamleiter:** Sieht nach der Anmeldung eine personalisierte Teamansicht mit den Daten des eigenen Teams und eine Übersicht der eigenen Fahrer. Teamleiter können Teamdaten und Fahrer bearbeiten, jedoch ausschließlich innerhalb ihres eigenen Teams.
- **Administrator:** Hat Zugriff auf alle Tabs der Anwendung. Dazu zählen die Verwaltung von Benutzern und Rollen, die Verwaltung aller Teams und Fahrer, die Einsicht und Bearbeitung aller Versuche, die Konfiguration der Bewerbe sowie der Export von Ergebnissen und Versuchen.

Tabs

Die Anwendung ist in folgende Tabs unterteilt, die je nach Rolle sichtbar sind:

- **Leaderboard:** Zeigt die Ergebnislisten aller Bewerbe, gefiltert nach Teamkategorie. Für jeden Bewerb wird ein eigenes Ranking mit Punktestand angezeigt.
- **Attempts:** Ermöglicht Administratoren die Einsicht, Bearbeitung und Löschung aller Zeitversuche. Start- und Endzeit sowie Energieverbrauch können nachträglich korrigiert und Versuche als ungültig markiert werden.
- **Teams:** Bietet Administratoren eine vollständige Übersicht aller Teams inklusive Fahrerliste. Teamleiter sehen hingegen ausschließlich ihr eigenes Team in einer detaillierten Einzelansicht.
- **Challenges:** Erlaubt Administratoren die Konfiguration der Bewerbe.
- **Users:** Ermöglicht Administratoren das Erstellen, Bearbeiten und Löschen von Benutzern sowie die Verwaltung von Rollenzuweisungen.
- **Export:** Stellt Administratoren den Export der Ergebnislisten und Zeitversuche als CSV- oder XLSX-Datei zur Verfügung, gefiltert nach Bewerb und optional nach Teamkategorie.

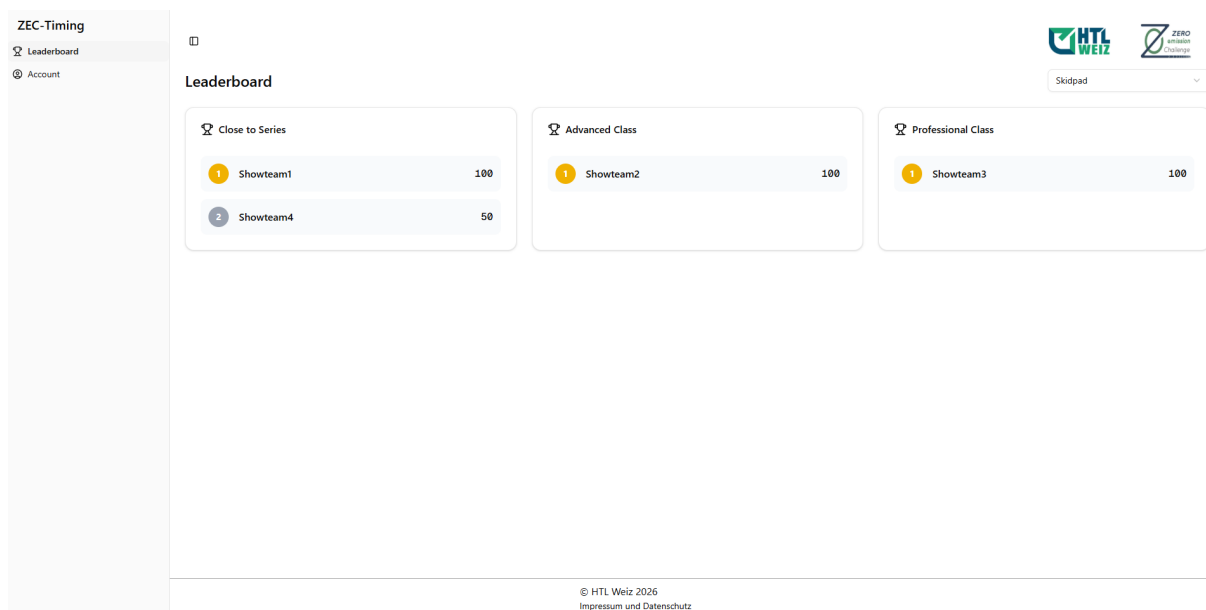


Bild 4: Leaderboard-Tab für Viewer

ZEC-Timing

Leaderboard

Teams

Account

Showteam1

Team Details

Category

Close to Series

Vehicle Weight

10 kg

Mean Power

10 W

RFID Identifier

123456789

Edit Team

Team Drivers

+ Add Driver

Simon

Weight: 10 kg

HTL WEIZ

ZERO emission challenge

© HTL Weiz 2026
Impressum und Datenschutz

Bild 5: Teams-Tab für Teamleiter

ZEC-Timing

Leaderboard

Attempts

Teams

Challenges

Users

Export

Account

Teams & Drivers Management

All Teams

Team Name	Category	Weight (kg)	Power (W)	RFID	Actions
Showteam1	Close to Series	10	10	123456789	

Drivers

Simon

Weight: 10kg

HTL WEIZ

ZERO emission challenge

© HTL Weiz 2026
Impressum und Datenschutz

Bild 6: Teams-Tab für Administrator

5 Implementation

5.1 Server

5.2 Web App

6 Testen

6.1 Teststrategie

Die Teststrategie des Projekts verwendet Unit- und Integrationstests für die Backend-Services so wie End-to-End-Tests für die Webanwendung. Als Testframework für die Backend-Tests wird `pytest`[15] eingesetzt, während für die End-to-End-Tests `Playwright`[16] zum Einsatz kommt. Zur Sicherstellung der als Ziel gesetzten Testabdeckung von über 80% wird `pytest-cov`[17] verwendet. Die Testabdeckung wurde in allen Backend-Services erreicht, im Durchschnitt mit 85%. Um diese zu erreichen, wurden nicht nur die "Happy PathSzenarien, sondern auch die Fehlerfälle getestet.

6.2 Pytest

Wie in 6.1 beschrieben, wurden die Backend-Tests mit `pytest` umgesetzt. Für die jeweiligen Tests wird eine SQLite-Testdatenbank[18] verwendet, um eine isolierte Testumgebung zu schaffen. Die Testdaten werden über `pytest-Fixtures`[19] bereitgestellt. Abhängigkeiten von anderen Services werden mit `unittest.mock`[20] bzw. `monkeypatch`[21] simuliert. So kann die Businesslogik der Services isoliert getestet werden.

6.2.1 Unit-Tests

Unit-Tests überprüfen die Funktionalität einzelner Funktionen oder Methoden innerhalb der Services. Sie stellen sicher, dass die Kernlogik erwartungsgemäß funktioniert.

Ein Beispieltest für die Funktion `get_fastest_attempt`:

```
1 def test_fastest_attempt(db, seeded_attempts):
2     attempt = crud.get_fastest_attempt(db=db, challenge_id=1)
3     time = attempt.end_time - attempt.start_time
4     assert time == timedelta(seconds=9)
```

6.2.2 Integrationstests

Integrationstests prüfen die HTTP-Schnittstellen des Services mit dem FastAPI `TestClient`[22]. Diese Tests prüfen das Verhalten der API aus Sicht eines externen Clients, indem sie HTTP-Anfragen an die Endpunkte senden und die Antworten validieren.

Ein Beispieltest für das Abrufen aller Attempts:

```
1 def test_list_attempts(client):
2     response = client.get("/api/attempts/")
3     assert response.status_code == 200
4     assert len(response.json()) == 2
```

6.3 Playwright

Wie in 6.1 beschrieben, werden die End-to-End-Tests mit Playwright[16] umgesetzt. Ziel dieser Tests ist die Validierung der Webanwendung aus Sicht eines realen Benutzers im Browser. Im Unterschied zu den Backend-Tests, welche die Funktionalität isoliert testen, wird hier das Gesamtsystem getestet.

Ein Beispieltest dafür, ob ein Admin-Benutzer alle Tabs der Webanwendung sehen kann:

```
1 test('admin can log in and see all tabs', async ({ page }) => {
2     await loginAs(page, 'admin', 'changeme');
3
4     await expect(page.getByRole('button', { name: 'Leaderboard' })).toBeVisible();
5     await expect(page.getByRole('button', { name: 'Attempts' })).toBeVisible();
6     await expect(page.getByRole('button', { name: 'Teams' })).toBeVisible();
7     await expect(page.getByRole('button', { name: 'Challenges' })).toBeVisible();
8     await expect(page.getByRole('button', { name: 'Users' })).toBeVisible();
9     await expect(page.getByRole('button', { name: 'Export' })).toBeVisible();
10 })
```

7 Fazit und Ausblick

7.1 Zusammenfassung

In dieser Diplomarbeit wurde ein Timing-System für die ZEC entwickelt. Das System bietet eine REST-API Schnittstelle, die es ermöglicht, Versuche in die Datenbank einzutragen, welche auch automatisch bewertet werden. Zusätzlich können die Ergebnisse über die API abgerufen werden, und alle wichtigen Informationen können einfach über die Weboberfläche eingesehen und bearbeitet werden.

Der Microservice-Ansatz hat sich als passend für das Projekt erwiesen, da er eine klare Trennung der Verantwortlichkeiten ermöglicht und die Wartbarkeit des Systems verbessert. Die Verwendung von Docker-Containern war ebenfalls eine richtige Entscheidung, da so die Bereitstellung erleichtert wurde und eine Unabhängigkeit von der Infrastruktur gewährleistet werden konnte. Die Verwendung einer zentralen Datenbank hätte die Entwicklung erleichtert, allerdings hätte dies die Eigenständigkeit der Services massiv eingeschränkt, weshalb die Entscheidung für einzelne Datenbanken pro Service auch hier die richtige war.

7.2 Zukunftsausblick

Zusätzlich zu den offensichtlichen Erweiterungen, wie der Fertigstellung der noch nicht implementierten Features, etwa der Desktop-App zur Verbindung mit der Lichtschranke per MQTT, der Kubernetes-Integration und den Continuous Integration / Continuous Deployment (CI/CD)-Pipelines, kamen während der Entwicklung auch andere Ideen auf, die in Zukunft umgesetzt werden könnten.

- **SD-Karten-Schutz:** Derzeit ist kein Schutz für die SD-Karten der Raspberry Pis implementiert. Eine Implementierung von Caching im Backend oder Frontend wäre notwendig, sollte der Server auf einem Raspberry Pi laufen.
- **Anbindung einer Bluetooth-Datenübertragung:** Im derzeitigen System muss der Zeitnehmer von den Sensoren ablesen, wie viel Energie verbraucht wurde. Eine Anbindung der Bluetooth-Datenübertragung könnte es ermöglichen, dass die Sensoren die Daten an die Desktop-App übermitteln.